Payload manifest (L1)

TBK SPEC 03:2026, Version 1.0

The Tebako Project

Unrestricted
November 01, 2026
TBK SPEC 03:2026, Version 1.0
Tebako
© 2026 The Tebako Project. All rights reserved

Ribose

Contents

Foreword 5

1. Location and form 5

2. The three blocks 5

2.1 IDENTITY (all kinds) 6

2.2 PROVIDES (kind-specialized) 6

2.3 DEPENDS (`requires:`) 9

2.4 MATERIALIZE (`materialize:`, additive — schema_minor 1) 11

2.5 LIBRARY ALIASES (`library_aliases:`, additive — schema_minor 2) 12

2.6 CHECKS (`checks:`, additive — schema_minor 3) 12

2.7 INTERP ENV (`interp_env:`, additive — schema_minor 8) — IMPLEMENTED 2026-10-07 (tebako#559) 13

2.8 AUGMENTS (`augments:`) and EXTENSION POINTS (`extension_points:`) — additive, schema_minor 10 14

2.9 MIN RUNTIME TEBAKO (`min_runtime_tebako:`, additive — schema_minor 15, tebako#666) 17

2.10 ENV PASS (`env_pass:`, additive — schema_minor 16, tebako#737) 17

© 2026 The Tebako Project

3. Platform axis (locked, vcpkg-triplet form) 18

4. The three tiers (no duplicated authority) 19

5. Production and consumption 19

6. The package manifest (L2, extension block type 2) 20

7. The digest fixed-point rule (locked 2026-07-26) 22

8. Executable dependency edges (locked 2026-07-27) 23

© 2026 The Tebako Project

All rights reserved. Unless otherwise specified, no part of this publication may be reproduced or utilized otherwise in any form or by any means, electronic or mechanical, including photocopying, or posting on the internet or an intranet, without prior written permission. Permission can be requested from the address below.

The Tebako Project

Suite 1, 8/F, New Henry House
10 Ice House Street
Central
Hong Kong

copyright@ribose.com
www.ribose.com

 © 2026 The Tebako Project

Foreword

Normative specification of the in-image manifest: every payload is a **self-describing artifact**, not a blob. Status: SHIPPED (the block + the engine shipped; the §3 press/install gate wiring remains — the checks block's discoverability row).

1. Location and form

- Path: `/__tpkg__/manifest.yaml` — inside the image, at a well-known location. No sidecars, no central service.
- Format: **YAML** (the locked convention for all authored configuration surfaces). The versioned grammar: `docs/spec/schemas/payload-manifest.yaml` (the registry's authored form; `crates/tpkg/src/manifest.rs` mirrors it).
- Integrity: the manifest is inside the image, so it is exactly as tamper-proof as the content and is covered by the image's digest and signature. Any consumer (bootstrap, dispatcher, `tfs` CLI, a foreign project) reads it through TFS itself.

2. The three blocks

Every manifest is **IDENTITY + PROVIDES + DEPENDS** on a common provenance/trust layer.

© 2026 The Tebako Project

2.1 IDENTITY (all kinds)

FIGURE 1

```
schema_version: 1
kind: app | data | toolkit | runtime | language
name: metanorma
version: 1.2.3
producer: {tool: tebako-cli, tool_version: 0.16.0}
created: 2026-07-26T00:00:00Z
source: {src_sha256: "…", commit: "…", builder: "gha:
run:123"}
sbom: {ref: "…"}                        # optional
digest:
  tree_hash: "…"                        # plaintext merkle
root over the tree MINUS the
                                        # manifest path —
semantic identity (CAS)
  blob_sha256: "…"                      # transport identity —
see the fixed-point rule
signing: {state: unsigned | signed, keyid: "…", mechanism:
openpgp}
encryption: {state: none}               # or per-part list
{paths, algorithm, envelope_refs} — NEVER keys
annotations: {…}                        # free-form; unknown
keys preserved
```

2.2 PROVIDES (kind-specialized)

app (executable):

FIGURE 2

```
entrypoints:                            # ARRAY — multi-entry
suites; N=1 for simple apps
  - name: metanorma                     # the command name
(shim registers under this)
    path: /app/bin/metanorma            # inside the image
    args_default: []                      # interpreter-side
defaults composed BETWEEN interpreter
                                          # and entry by the
runtime driver (spec 17 §1 — the single
                                          # owner, tebako#503)
— java: ["-jar"]. A ZERO-runtime entry
                                          # has no driver: the
shim appends them as the program's
                                          # leading args.
    runtime_requirement: {engine: ruby, constraint: ">= 3.3,
< 5.0"}
```


© 2026 The Tebako Project

```
      # OPTIONAL — omit entirely for native entrypoints (see
below);
      # range for pure-language; abi-line "~> 3.3.0" for
native-extension payloads.
      # engine names the LANGUAGE — mri/jruby/truffleruby
are all `ruby`;
      # the OPTIONAL `implementation:` narrows to one (spec
28 §8).
      # The value may ALSO be a LIST of such maps — `any_of`
semantics,
      # OR in declaration order — when the admissible set
differs per
      # implementation (spec 28 §8); an entry without
`implementation:`
      # matches at the language level (the runtime's
language_version).
    # native-extension entrypoints ALSO pin the
implementation and the
    # platform line:
    runtime_requirement: {engine: ruby, implementation: mri,
constraint: "~> 3.3.0", abi: "arm64-darwin-23"}
      # abi = the runtime's own platform string the
extensions were built
      # against (ruby: Gem::Platform.local.to_s). The
version line and the
      # platform line are ORTHOGONAL — resolution checks
both (spec 05 §5).
      # an `abi:` in force REQUIRES `implementation:` — an
ABI is
      # per-implementation by construction (spec 28 §8).
  platforms: [x86_64-linux-gnu, aarch64-macos]  # native-ext
apps are triplet-bound;
                                                # universal
only for pure-language
capabilities: {exec: true, read: true}
```

`capabilities` may also carry **`host`** (any kind): the host-access jail the payload was built to need — a REQUEST the dispatch surfaces compose with the user's tightening, never a grant to itself. spec 08 §4 is normative for its shape (`default`, `mounts`, `argument_files`).

Zero-runtime entrypoints (locked): an entrypoint whose executable is native (or self-contained) needs NO interpreter payload — inkscape-class slices, static binaries, shell-free tools. Such apps declare no `runtime_requirement` and the dispatcher mounts zero runtime payloads. Runtimes (ruby today; python, julia, others tomorrow) are just one payload kind among many — never an assumed layer. A SUITE is one package with N entrypoints — the shim layer exposes one command per entry, each dispatching to its own image and its own runtime requirement (spec 07).

Completeness (locked 2026-08-29): `entrypoints` declares EVERY command the image carries that a user may invoke directly — it is the payload's command declaration, not a primary-executable field. A bundled CLI (a font manager, a bibliography tool) is as much a

© 2026 The Tebako Project

provided command as the app's namesake. The declaration is the COMPLETE inventory; exposure is an orthogonal per-entrypoint key:

FIGURE 3

```
    - name: fontist
      path: /bin/fontist
      active: false     # OPTIONAL (default true) — declared
but NOT
                        # PATH-registered at install
```

`active` is the payload author's default exposure. Install registers only the active set (spec 07 §1); an inactive-but-declared command stays dispatchable — `tebako shim enable <name>` links it on demand, `tebako run <name>` reaches it directly, and the user's per-machine enable/disable always overrides the manifest default (spec 07 §3). Additive on the wire: pre-flag readers ignore the key and register every entrypoint (the pre-flag behavior). The declared names are also the only set a self-contained package's `entries[]` may draw from (§6): the L2 entry names mirror the L1 declaration — one SSOT, cross-checked by `tebako-pkg validate` (tebako#494).

runtime (provides an interpreter):

FIGURE 4

```
provides: {engine: ruby, implementation: mri, version: 4.0.6,
 language_version: "4.0", abi_line: "4.0", platform: x86_64-
linux-gnu}
  # engine = the LANGUAGE; implementation (REQUIRED for
kind: runtime) =
  # which ruby this is (mri/jruby/truffleruby — spec 28 §8).
  # version = the implementation's OWN version line (jruby:
9.4.8);
  # language_version = the language level it implements
(jruby 9.4 → "3.1";
  # for mri the two are identical). spec 05 §5 matches on
both.
built_from: {src_sha256: "…", patch_set: "v0.2.8"}
env: {GEM_PATH: …}
entrypoints:                        # OPTIONAL (additive —
schema_minor 4, spec 30 §2):
  - name: jing                      # the commands this
runtime boots as a SEPARATE
    path: /bin/jing.jar             # process for a
consumer's `kind: runtime` edge
    args_default: ["-jar"]          # `expose:` list — the
app-entrypoint grammar minus
                                    # runtime_requirement
(a runtime runs on itself).
                                    # Absent/empty = the
runtime serves only as a
```


© 2026 The Tebako Project

```
                                        # primary co-mounted
runtime.
windows_boot: materialize               # OPTIONAL (additive —
schema_minor 11, spec 17 §7):
                                        # the runtime's
interpreter cannot consume the
                                        # driver's mounted
images on windows (zero-patch,
                                        # no interposition
tier) — boot from the extracted
                                        # exec-cache tree
instead. Absent = the interpreter
                                        # reads the mounts
directly (ruby). POSIX never reads it.
capabilities: {exec: true, read: true, runtime: true}
```

data (read-only):

FIGURE 5

```
mount_semantics: {suggested: /usr/share/fonts}
consumers: [any]
capabilities: {exec: false, read: true}
```

toolkit (native layer, e.g. gtk/qt): as runtime, minus the interpreter contract; PROVIDES names libraries/tools other payloads DEPEND on.

2.3 DEPENDS (`requires:`)

FIGURE 6

```
requires:
  - kind: language              # a language runtime edge
    engine: ruby
    implementation: jruby       # OPTIONAL — absent: any
implementation of the
                                # engine whose language_
version satisfies the
                                # constraint (spec 28 §8)
    constraint: "~> 3.3.0"
  - kind: toolkit               # a native toolkit layer
    name: gtk-layer
    constraint: ">= 3.24, < 3.25"
    triplets: [aarch64-macos, x86_64-linux-gnu]   # resolves
on these hosts
                                                  # only —
skipped elsewhere
```


© 2026 The Tebako Project

```
                                                    # (the per-
edge rule below)
    mount: /__layers__/gtk
  - kind: data
    name: iso-codes
    constraint: ">= 2024.1"
    mount: /__app__/share/iso-codes
  - kind: runtime             # a SPAWNED runtime edge
(additive — schema_minor 4,
    engine: java              # spec 30): resolves through
the RUNTIME index into the
    implementation: temurin   # OPTIONAL (spec 28 §8)
    constraint: ">= 21"       # store's runtimes/ area and
is NEVER co-mounted — its
                              # wrapper exe executes FROM
THE STORE.
    expose: [java, keytool]   # OPTIONAL — the depended
entries this payload surfaces
                              # (spec 30 §3's shim surface)
```

- Dependency edges may also name **provided executables** (e.g. an app requiring the `inkscape` command resolves against the PROVIDES of the inkscape payload) — the provides/requires graph resolves capabilities, not just payload names.
- A **`kind: runtime`** edge is spec 30's spawned dependency: the depended runtime is dispatched as a CHILD process through its store-resident wrapper (never co-mounted, never materialized); resolution placement, the spawn wire, the jail union, and the shim surface are spec 30 §2–§5's. EXCEPTION: when the DECLARING payload is itself `kind: runtime` and the edge carries NO `expose:`, the edge is an OWNER edge — runtime-on-runtime composition (spec 33): the depended runtime owns the boot's process, and the declarer contributes a second env image plus the `on_runtime` argv template.
- **MOUNT RULE (locked):** the mount point is declared in the CONSUMER's manifest (docker-compose volume semantics): the consumer's code knows where it looks for things; the provider never dictates its mount location. The dispatcher resolves the capability, then mounts at the consumer-declared path — top-level or inside the app's memfs namespace.
- **Resolution algorithm:** read manifest → build the graph → topological order (runtime first, layers, app last) → per node: cache hit on (constraint × triplet) → use, else fetch (signature-verified, spec 09) → verify → cache (0444 + markers) → compose the mount stack with the manifest's declared env → exec. The SAME signed `.tfs` artifact type at every graph level — one coherent algebra.
- **PER-EDGE PLATFORM CONDITIONING (`triplets:`, additive — schema_minor 9, locked 2026-09-12, roadmap 86):** every edge kind EXCEPT `kind: language` accepts the OPTIONAL `triplets:` list — the §3 axis's vcpkg-form triplets, non-empty, no duplicates (validated at parse). This generalizes the toolkit edge's existing spelling to `data`, `runtime`, and `executable` edges — one spelling per axis (spec 00 invariant 10): payload-level coverage is `platforms:`, edge-level conditioning is `triplets:`.
  - **Absent = universal.** An edge without `triplets:` resolves on every host — byte-identical with pre-minor-9 behavior.

© 2026 The Tebako Project

- A host outside the list SKIPS the edge: no resolution, no fetch, no install row, no lock row, no mount, no spawn surface, no exposed-name registration. The skip is LOUD — one journal line (`event=edge-platform-skip`, the shim's journal convention) plus a stderr note at press / install / dispatch — never silent, never an error. A multi-host compose (spec 23) evaluates coverage per target host: the edge lands in the lock rows of the covering hosts only.
- A skipped edge's exposed commands are absent from the shim surface on the skipping host; dispatching one anyway (a stale shim, a hand-written invocation) is a named error — "not available on this platform", exit 69 — never a silent fallback to another provider.
- **`kind: language` FORBIDS the key** — a skipped language edge would boot a payload with no runtime, an incoherent composition; the payload-level `platforms:` axis already owns "where this payload runs". A language edge carrying `triplets:` is a named manifest error at parse, steering the author to `platforms:`.
- **`critical:` interaction:** the skip applies regardless of `critical: true` — `critical` governs reader-ERA refusal (a pre-minor reader must refuse an edge it cannot interpret), never platform reach. "This host must have the dep" is said with `platforms:` on the payload, not with `critical` on the edge.
- Reader compatibility: a pre-minor-9 reader ignores `triplets:` on non-toolkit edges (the unknown-field rule) and resolves the edge universally — the safe direction: an extra dependency resolved, never a needed dependency skipped.

2.4 MATERIALIZE (`materialize:`, additive — schema_minor 1)

FIGURE 7

```
materialize:
  - /lib/ssl/certs/cacert.pem    # an absolute in-image
path, no '..' components
```

A top-level list (any kind may declare it; old readers ignore it under the unknown-field rule) naming **regular files a C library must read through its own IO** — the interpreter's patched IO never gets asked, so the bytes must exist on the host filesystem. The OpenSSL CA cert is the canonical entry. The driver extracts each declared path after the mounts and the jail, before the interpreter handoff, to `<TEBAKO_EXEC_CACHE>/resources/<image-key>/<P>` — whole-file, read-only, digest-verified. A declared path absent from the image, or not a regular file, is the manifest lying: boot fails by name (exit 65), never a skipped entry. The full mechanics and the trust chain are spec 22 §4 (class R); the grammar is registered in `docs/spec/schemas/payload-manifest.yaml`.

© 2026 The Tebako Project

2.5 LIBRARY ALIASES (`library_aliases:`, additive — schema_minor 2)

FIGURE 8

```
library_aliases:
  - name: libfoo-3.dll        # the exact bare name a loader
call presents —
                              # no path separator, no drive
qualifier
                              # (validated at parse)
    path: /lib/libfoo-3.dll   # the in-image absolute file
the name resolves to
```

A top-level list (any kind may declare it; old readers ignore it under the unknown-field rule) naming the ONLY bare library names that resolve to the image's own files — the declarative half of the windows Class-L bare-name rule (semantics: spec 22 §2.1). A loader call presenting a bare name matching no entry is a HOST reference and passes through untouched: host-by-default, no probing, no heuristics, no silent fallback. Matching is verbatim and case-insensitive (the windows loader's own comparison); the name is never extension-completed (`foo` does not match `foo.dll`). `path` must be an absolute in-image path of a regular file, no `..` components — an absent or non-file target is the manifest lying (a named 65 at boot, the §2.4 precedent). A duplicate `name` within one image is a named manifest error; two co-mounted images declaring the same `name` is a named boot error (authoring ambiguity, the spec 17 §2 slug precedent), never a silent winner. No platform filter: native images are triplet-bound (§3), so an alias is platform surface by construction. The grammar is registered in `docs/spec/schemas/payload-manifest.yaml`.

2.6 CHECKS (`checks:`, additive — schema_minor 3)

FIGURE 9

```
checks:
  html-xml:                       # the check name — [A-Za-
z0-9][A-Za-z0-9._-]*
    entry: /bin/metanorma         # in-image executable;
"self" on kind
                                  # runtime only (the
runtime exe itself)
    argv: ["--type", "iso", "{scratch}/test-iso.adoc", "--
agree-to-terms"]
    fixtures: /__tpkg__/check/html-xml   # exec-only;
CONTENTS land at the
                                         # host scratch root
    expect: {exit: 0, files: ["test-iso.xml", "test-iso.
html"]}
    timeout: 180
```


© 2026 The Tebako Project

```
  layout:                              # no entry ⇒ STRUCTURAL
(the data-slice
    expect:                            # shape): the engine
mounts and asserts
      image_files: [/templates/org/cover.adoc]   # in-image,
exist + non-empty
```

A top-level map (any kind may declare checks; old readers ignore it under the unknown-field rule) naming the payload's own acceptance contracts — "given my declared needs, I do my one real thing". ONE key decides the shape (MECE, never a `kind:` flag): `entry` present ⇒ an exec check; absent ⇒ a structural check, which declares no `argv`/`fixtures` and asserts via `expect.image_files`. `expect.files` are scratch-relative existence + non-empty assertions; byte-golden assertions do not exist by construction. A malformed block is a named validation error at press / `tfs validate` (exit 65), never discovered at run time. The semantics — the engine, the three moments, SKIP/FAIL discipline — are spec 26; the grammar is registered in `docs/spec/schemas/payload-manifest.yaml`.

2.7 INTERP ENV (`interp_env:`, additive — schema_minor 8) — IMPLEMENTED 2026-10-07 (tebako#559)

FIGURE 10

```
interp_env:
  RUBY_YJIT_ENABLE: "1"          # interpreter-option
defaults, literal strings
  RUBY_GC_HEAP_FREE_SLOTS: "500000"
```

A top-level map (any kind may declare; old readers ignore it under the unknown-field rule) carrying interpreter-option defaults the slice author owns: what options the interpreter starts with is a packaging-time decision (the runtime/payload slice developer's), with the local user always able to override. The dispatcher exports each merged key into the runtime process's environment at dispatch, so the interpreter reads it at its own boot (ruby's `RUBY_YJIT_ENABLE`, python's `PYTHON_JIT`, the JVM family's option vars — the channel is generic, never a ruby wart).

MECE against the existing env surfaces: `provides.env:` (runtime kind) is the runtime's COMPOSED process-env defaults (path-class included, spec 07 §9); `interp_env:` is the per-slice interpreter-OPTION channel — scalar only, never path-class, never a `TEBAKO_*` control var (those are dispatcher/driver-owned; both violations are named manifest errors). Values are literal strings: no expansion, no execution, no references — additive and declarative by construction; a malformed block is a named validation error at press / `tfs validate` (exit 65), never discovered at run time.

The composition chain and the override order live in spec 07 §9; the driver-side application rule in spec 17 §2.2; the audit surface in spec 15 §4. A declaration is a DEFAULT, never a

© 2026 The Tebako Project

requirement: a pre-minor-8 consumer ignores the key and ships the interpreter's built-in behavior.

2.8 AUGMENTS (`augments:`) and EXTENSION POINTS (`extension_points:`) — additive, schema_minor 10

The reverse dependency edge: a slice that extends a base payload it cannot be enumerated BY. The mount rule (§2.3) is unchanged — the CONSUMER declares; here there are two consumers and both keep their say: the BASE declares WHERE extensions land, the SLICE declares WHICH base it extends. Neither touches the other's bytes.

Base side — PROVIDES gains `extension_points:` (any kind that can host in-process content; practically app):

FIGURE 11

```
provides:
  entrypoints: [...]
  extension_points:
    - name: flavors              # symbolic; slices
reference it by name
      mount: /flavors.d          # VFS-absolute; the slices
mount BELOW it
      layout: gem-home           # enum: gem-home | files
(extensible)
```

- `layout: gem-home` — each mounted child is a mini GEM_HOME (`gems/`, `specifications/`); composition is the language ecosystem's own multi-home search list (GEM_PATH), never a union mount: two slices carrying different versions of one gem stay VISIBLE-and-resolvable (requirement-based activation) instead of one shadowing the other by mount precedence.
- `layout: files` — opaque content the application reads itself (fonts, codelists). The app enumerates `<mount>/<slice>/...`.

Base side, dedup inventory — PROVIDES gains `gems:` (any payload staging a gem home; REQUIRED when it declares a `gem-home` extension point):

FIGURE 12

```
provides:
  gems:
    - {name: nokogiri, version: "1.18.9"}
    - {name: metanorma-cli, version: "1.16.9"}
```


© 2026 The Tebako Project

- Generated, never hand-authored (spec 00 §10): the builder scans the staged gem home's `specifications/*.gemspec` before the image seals. One generator, two consumers — the dispatcher reads the manifest key (the overlap journal, spec 07 §2 step 3a); the release exports the same inventory as its `closure.json` asset, the slice press's subtraction source. A second hand-written copy anywhere is a bug.

Slice side — a new top-level block `augments:` (sibling of `requires:`):

FIGURE 13

```
augments:
  - payload: metanorma            # the base payload's name
    constraint: "= 1.16.2"        # vs the base's RESOLVED
version
    extension_point: flavors      # names the base's declared
point
    built_against:                # provenance, written by
the press
      version: "1.16.2"           #   tooling — the exact
base release
      closure_sha256: "…"         #   whose inventory was
subtracted
```

`augments` IS a dependency declaration, and the slice is NOT self-contained by construction (a deduped slice's gems resolve against the BASE's gem home at run time). Dispatch semantics live in spec 07 §2 step 3a; the store scan in spec 05 §3. The scope locks:

- **Root-only.** `augments` attaches slices to the DISPATCHED (root) payload. A slice augmenting a slice is the named NestedAugment error; `augments` on a root payload's own manifest is ignored (journaled `slice-ignored`) — attachment is declared by the slice, never claimed by the base.
- **Content-only.** Slices are in-process CONTENT (gem homes, files) — `capabilities.exec: false`. An extension carrying executables rides the forward `requires`/`expose` machinery (spec 30), never `augments`. MECE.
- **No runtime of its own.** The slice's `requires` language edge is satisfied by the BASE's resolved runtime (abi line included — a mismatch is the dispatch-time loud skip, never a segfault).
- **The binding rule.** A slice deduped against the base's closure pins EXACTLY (`constraint: "= x.y.z"` — one slice release per base release, shipped in lockstep): subtraction is sound only against the exact inventory it subtracted, and the slice's tests ran against exactly that base. When the base ships runtime LINES as suffixed versions (spec 28; `1.16.9` and `1.16.9-ruby4.0` of one payload version), the closures differ per line, so the pin names the LINE's exact version (`= 1.16.9-ruby4.0` — the schema_minor-12 grammar, spec 05 §5) and one slice release exists per (slice, base LINE) pair. Range constraints remain legal for content that honestly spans (`layout: files` asset packs); gem-closure slices MUST pin exactly. Pre-minor-10 readers ignore

© 2026 The Tebako Project

both keys (the slice then simply attaches nothing — loud at install, never silent); pre-minor-12 readers reject a suffixed pin by name (exit 65). **Family completion:** when the lines' closure INVENTORIES are identical (press-verified at build time against every family row's published closure assets — never assumed), ONE slice release MAY serve the whole family: it carries one exact-pin `augments` edge per family row (each with its own `built_against` provenance naming that row), and the dispatcher attaches when any edge matches (spec 07 §2 step 3a). The binding rule is unchanged — every edge pins exactly the row whose inventory the press verified identical to the subtraction source.

The line-split idiom, worked (first production use: the metanorma flavor slices, metanorma/packed-metanorma-bsi and -nist, 2026-09-27 — the living instance to copy). A pure-ruby slice that serves every runtime line of a base family cannot use ONE exact pin: equality clauses are suffix-exact in the versions grammar (`= 1.17.0` never matches `1.17.0-ruby3.3`). Nor should it reach for a range (`>= 1.17.0-ruby3.3, < 1.17.1`): a range is legal for content that honestly spans, but a gem-closure slice subtracts a specific inventory, and the binding rule wants each served row named. The shipped spelling is one edge per row:

FIGURE 14

```
augments:
  - payload: metanorma
    constraint: "= 1.17.0"
    extension_point: flavors
    built_against: {version: "1.17.0", closure_sha256:
"…"}
  - payload: metanorma
    constraint: "= 1.17.0-ruby3.3"
    extension_point: flavors
    built_against: {version: "1.17.0-ruby3.3", closure_
sha256: "…"}
```

Soundness: the press subtracts the INTERSECTION of the family's platform/line closures — a gem is subtracted only if present in every closure the slice will mount beside — and refuses (build error) when the inventories are not identical. One subtraction therefore serves every named row; the dispatcher's any-edge match (v2.8.19) attaches the slice to whichever row the user's version pin selected. A new base line (say `-ruby3.5`) is a NEW row the slice does not name: dispatch skips loudly (the unattached-flavor journal), and the slice press adds the edge after verifying the new row's closure identity — never a silent range stretch.

© 2026 The Tebako Project

2.9 MIN RUNTIME TEBAKO (`min_runtime_tebako:`, additive — schema_minor 15, tebako#666)

An OPTIONAL top-level key naming the minimum tebako tooling version of the runtime this payload runs on:

FIGURE 15

```
min_runtime_tebako: "2.8.8"
```

The author sets it when the payload starts depending on a newer runtime/driver capability (a mount-ancestor route, a spec-32 `kind: executable` edge, ...). The value is 1–4 dot-separated components of the schema_minor-12 component grammar (leading decimals, optional `-label` suffix); anything else is a named manifest error at parse. Absent = no floor. Old readers ignore the key (the unknown-field rule) — which is exactly why the key exists: the failure mode it retires is a stale cached runtime failing a newer payload with payload-blaming wording.

Semantics (the three consumers, all fail-closed with named errors):

- Resolution (spec 05 §5): "newest compatible cached" discards cached runtimes whose own tebako version is below the floor and fetches instead; the miss errors NAME the skipped entries. `TEBAKO_OFFLINE=1` keeps fail-closed behavior — cache-or-named-error, with the floor named. A downloaded runtime below the floor (a pinned stale line) is the named stale-runtime refusal.
- The driver (spec 17 §8): a runtime whose tooling version is below the floor refuses the manifest by name — "the runtime is too old for this payload's manifest (needs X, have Y)" — both when the manifest parses (the key itself is old-reader-safe) and when the parse FAILED (the manifest uses grammar this runtime predates; the floor is then read by a tolerant top-level scan of the raw YAML). Never "corrupt manifest / self-description lies" for the stale-runtime class.
- The floor constrains every runtime the payload's dispatch resolves — its own entrypoint runtime and the spawned runtimes of its `requires` edges (a provider payload's floor likewise gates the provider's own pair).

The floor is a MINIMUM, not a pin: it never narrows which compatible runtime wins above it, and it never substitutes for the `runtime_requirement` constraint (the interpreter version axis is the requirement's; the tooling axis is the floor's).

2.10 ENV PASS (`env_pass:`, additive — schema_minor 16, tebako#737)

The packager's opt-in to the driver's credential scrub. A packaged app must not exfiltrate the packager's credentials by default: at boot's end, after every contract read, the driver BLANKS every `TEBAKO_*` environment variable that is not part of the boot contract (spec 17 §1/§7's handoff surface, spec 08 §2's jail, spec 22 §6's exec cache, spec 22 §3's child

© 2026 The Tebako Project

injection, spec 25 §2's trace, and the TEBAKO_MOUNT_* discovery prefix) — TEBAKO_GITHUB_TOKEN above all. The scrub journals each NAME at debug, never a value.

FIGURE 16

```
env_pass:
  - TEBAKO_FLAVOR_KEY
```

- Every entry is a TEBAKO_-prefixed variable name (uppercase alphanumerics and underscore); anything else is a named manifest error at parse. The opt-in may only name what the scrub itself could take.
- The driver reads the list from the FIRST image's embedded manifest (the app payload); an absent/unreadable manifest means NO opt-in — the scrub fails safe. A standalone boot (no images) scrubs with no opt-in.
- Old readers ignore the key; the scrub is driver behavior, not a resolution input — the failure mode the key retires is a packaged run silently carrying the press host's credentials.

3. Platform axis (locked, vcpkg-triplet form)

platforms is EITHER "universal" (pure-ruby/data) OR an explicit list:

- aarch64-macos, x86_64-macos
- x86_64-linux-gnu, aarch64-linux-gnu
- x86_64-linux-musl, aarch64-linux-musl
- x86_64-windows-ucrt, aarch64-windows-ucrt

(aarch64-windows-ucrt joined the axis at payload-manifest schema_minor 13; pre-13 readers reject it by name — exit 65 — which is the designed fail-closed behavior for a triplet outside an old reader's axis.)

ONE Platform type (tpkg crate) owns the triplet ↔ release-asset-name mapping (aarch64-macos ↔ macos-arm64, x86_64-linux-gnu ↔ linux-gnu-x86_64, x86_64-windows-ucrt ↔ windows-ucrt64, ...) — dispatcher, release tooling, and registry consume the SAME mapping.

© 2026 The Tebako Project

4. The three tiers (no duplicated authority)

1. In-image manifest (this spec) — THE rich layer; signed/encrypted with the image.
2. tpkg trailer (spec 02) — stays minimal: mount/exec directives, per-slot digests, signature; REFERENCES manifests by image digest, never duplicates them.
3. Registry manifest (`tpkg-registry.yaml`, spec 04) — MIRRORS only resolution-relevant fields (kind, name, version, constraints, platforms, entrypoints) so the dispatcher resolves without downloading every payload.

5. Production and consumption

- `tebako press` / `tfs mkimage` embed the manifest at build time (kind inferred: app for pressed apps; data for plain images; runtime for runtime packages — `provides` filled from the factory's versions data).
- Payload exclusion at imaging time (additive, 2026-10-09, tebako#333): `tebako press --exclude <glob>` (repeatable) and `tfs mkimage --exclude <glob>` leave matching paths out of the BUILT image — one grammar, one semantics, one implementation shared by both commands. A pattern matches the payload-root-relative path (slash-separated, no leading `/`): `*`, `?`, and `[...]` classes follow fnmatch flag-0 semantics (a `*` run spans `/`), `\` quotes the next character, and a matched directory prunes its whole subtree (a trailing `/` on the pattern is accepted and stripped — it names the directory). Exclusion is imaging-time only: excluded paths are absent from the image, and the manifest never records the list (the image simply lacks them). The feedstock Tebakofile's `exclude:` list (spec 13 §9) is the same grammar, handed to the imager verbatim.
- `tfs info` prints the manifest; `tfs stat` exposes digests.
- The dispatcher (spec 07) resolves runtime compatibility FROM the manifest (`runtime_requirement` vs cached runtimes' `provides`).
- The bootstrap reads entrypoint/requirement from the manifest when present; the trailer stays the minimal path.

© 2026 The Tebako Project

6. The package manifest (L2, extension block type 2)

Locked 2026-07-26 (OCI model: manifest beside the layers, readable without backend knowledge — spec 02 §5b). The package manifest is a YAML extension block in the container, OUTSIDE every payload image. It owns composition; payload manifests (this spec) own self-description.

FIGURE 17

```
schema_version: 1
package: {name: metanorma, version: 1.2.3, producer: {…},
created: …}
entries:                          # one per invocable
command (N=1 for simple apps)
  - name: metanorma
    slot: 0                       # which payload image
    entrypoint: metanorma         # which PROVIDES
entrypoint inside it
    runtime_ref: ruby@3.4.2;tebako=0.15.9   # per-entry —
suites/multi-runtime
    interp_env: {RUBY_YJIT_ENABLE: "1"}     # the press-time
                                  # composition of slot 0's
L1 interp_env,
                                  # packager-refined (spec
07 §9's chain)
  - name: mn2pdf
    slot: 1
    entrypoint: mn2pdf
    runtime_ref: ruby@3.3.7;tebako=0.15.9
jail: {…}                         # package-level request
(spec 08)
env: {…}                          # package-level env
(composition rules: spec 07)
mounts:                           # per-slot mount semantics
(locked 2026-08-04)
  - slot: 0
    point: /__tfs__
    mode: union                   # exclusive (default) |
union; cow/enc reserved
    precedence: after-env         # union only: this image
shadows the env image
lock:                             # the press-time
composition lock (spec 23 §4/§13)
  runtime: {version: 0.16.19, carry: false}
  slices: [{name: metanorma, version: 1.2.3, carry: true,
slot: 0, sha256: …}]
  spawned:                        # spec 30 edges, press-
locked (spec 23 §13.6)
    - engine: java
      constraint: ">= 21, < 26"   # mirrored from slot 0's
L1 requires
```


© 2026 The Tebako Project

```
      expose: [java]              # mirrored from slot 0's
L1 requires
      version: "21.0.12"          # the press-time pick
      tebako: "2.1.5"
      carry: true
      exe:   {slot: 1, sha256: …} # carried pair — never
mounted
      image: {slot: 2, sha256: …}
```

- `runtime_ref` per entry kills the 128-byte single-field limit (suites, multi-runtime packages); the trailer's v1 field stays for v1 loaders.
- `entries[].name` / `entries[].entrypoint` mirror the slot payload's L1 `provides.entrypoints[]` (§2.2 completeness — one SSOT): every L2 entry names a DECLARED L1 entrypoint of its slot, enforced by `tebako-pkg validate` (tebako#494). Two entries MAY share one slot — same image, different in-image entrypoints: the multi-command single-payload form (one app slice carrying several CLIs, e.g. metanorma + fontist).
- `entries[].interp_env` (IMPLEMENTED 2026-10-07, tebako#559 — §2.7's L2 face): the press-time COMPOSITION of the slot payload's L1 `interp_env` with the packager's refinement (a packager key wins over the same L1 key; L1 keys pass through untouched otherwise). The mirror exists because the size-gated bootstrap reads no image bytes: the standalone dispatcher consumes exactly this block, never the in-image L1. `tebako-pkg validate` cross-checks the composition against the slot's L1 (every L1 key present with its declared or the refined value; a dropped L1 key is a named validation error, the tebako#494 class) — the block is a faithful mirror plus declared overrides, never an independent authority. Managed dispatch (registry payloads, no L2) reads the store's L1 manifest mirror instead (spec 07 §9's chain).
- v1-era packages without the block behave exactly as today (stub.rb / local conventions); the block is additive.
- `mounts` is optional; a slot without a `mounts` row mounts **exclusive** (spec 17's historical behavior — a duplicate point is a named error). `mode: union` merges the image over the images already mounted at `point` with declared precedence — read-only images only, directories merge, file conflicts resolve by precedence order (the env image is always lowest). `precedence` values: `after-env` (over the runtime's env image — the pressed-app form) or `after:<slot>` (over another payload slot). The union set is journaled at boot (spec 17 §1). `cow`/`enc` are RESERVED mode spellings on the same axis (the transforms law: overlays exist only in the Rust TFS) and are named errors until their specs land.
- `lock` is optional (absent on pre-spectrum packages — they resolve exactly as before). `lock.spawned[]` mirrors the app payload's L1 `requires[].kind: runtime` edges (engine/implementation/constraint/ expose) plus the press-time pick and carry verdict (spec 23 §13.6); `tebako-pkg validate` cross-checks the mirror against slot 0's L1 (the tebako#494 class), and the bootstrap resolves the rows at dispatch into the store's `runtimes/` area + `TEBAKO_SPAWN_LOCK` (spec 30 §2).

© 2026 The Tebako Project

toolkit (native layer, e.g. inkscape/gtk — the distro-ports model, spec 13 §9):

FIGURE 18

```
provides:
  executables: [{name: inkscape, path: /bin/inkscape,
version: 1.3}]
  libraries: [{name: xml2, path: /lib, abi: "2.12"}]
exec_tier: dynamic | wrapped | tfs-native | static   # spec
07 §8
exec_closure: [/bin, /lib, /share/inkscape]          #
static tier only
capabilities: {exec: true, read: true}
```

`exec_tier` tells the dispatcher HOW to run its executables (preload shim / link-wrapped / already-TFS-native / extract-closure) — consumers never care which path a tool took.

7. The digest fixed-point rule (locked 2026-07-26)

A manifest inside the image it describes CANNOT name that image's sha256 (self-reference has no fixed point). Therefore:

- `tree_hash` is computed over the payload tree EXCLUDING `/__tpkg__/` (manifest-excluded merkle root — well-defined, and it is what CAS/dedup/signing use). The construction (SHA-256, 4 KiB chunks, length-prefixed domain-separated serialization) is locked as `tfs-merkle-1` in `crates/tpkg/src/merkle.rs`; `tfs mkimage` and `tebako press` stamp it at image creation, `tfs info --verify` recomputes and compares (roadmap 37, SHIPPED; verification-on-READ inside the VFS is a later milestone).
- `blob_sha256` is NOT stored inside an embedded manifest: it lives one tier out — in the registry mirror (tier 3) and/or the tpkg trailer's per-slot digest array (tier 2, spec 02 §4). Producers fill it there; consumers verify the image against the OUTER record, never against a self-digest. An embedded manifest carrying a `blob_sha256` is read as advisory provenance only (e.g. the digest of the source image it was built from), never as a verification input (spec 15 §5).

© 2026 The Tebako Project

8. Executable dependency edges (locked 2026-07-27)

`requires` gains a fourth edge kind for the provides/requires capability graph (the inkscape case):

FIGURE 19

```
requires:
  - kind: executable          # an executable another
payload PROVIDES
    name: inkscape
    constraint: ">= 1.3"
    mount: /opt/inkscape      # consumer-declared, as always
```

Resolution: the dispatcher searches installed + available payloads for a PROVIDES.executables entry matching `name` + `constraint` (exact name, never a prefix/guess). Exactly one candidate → use it. Zero → named `DependencyNotFound(executable, constraint)` with a registry hint. More than one → named `AmbiguousProvider` listing candidates (payload, version, source registry) — the user pins with a payload-kind edge instead. The providing payload is mounted at the consumer-declared `mount`; its executables run per its own `exec_tier` (spec 07 §8).

The edge gains the OPTIONAL `expose:` list (additive — schema_minor 5, spec 32): an expose-carrying executable edge gains the SPAWNED payload surface — each exposed name dispatches the provider payload's own full spec-17 dispatch as a child process (its own runtime edge, its own mounts, its own jail union). `mount` and `expose` are orthogonal axes (VFS surface vs spawn surface; both may hold together), the capability match widens to `provides.executables` ∪ `provides.entrypoints[].name`, and every exposed name must resolve to a runtime-carrying entrypoint of the resolved provider (press/install cross-checked — a runtime-less match is a named error, never an exec-tier fallback). An exposed name never takes the exec-tier path — the spawn dispatch owns it. The child plan, the spawn-lock row, the lock's `spawned[]` payload row, the shim surface, the jail interaction, and the failure modes are spec 32 §2–§7's.

© 2026 The Tebako Project